A Multi-Objective Optimization Framework for CRISPR Guide RNA Design Using Machine Learning and Convex Optimization

Comprehensive Technical Report, Mathematical Proofs, Algorithm Architecture & Project Defense

Author: *Bioinformatics & Mathematics Optimization Team* | ***Date:*** *August 2026* | ***Target Reference:*** *Human Genome GRCh38.p14*

---

1. Project Overview & Biological Fundamentals

CRISPR-Cas9 (Clustered Regularly Interspaced Short Palindromic Repeats) is a revolutionary genome editing system adapted from bacterial adaptive immunity. The system relies on two critical components: the Cas9 endonuclease enzyme (which acts as molecular scissors) and a synthetic single-guide RNA (gRNA). The 20-nucleotide spacer sequence at the 5' end of the gRNA directs Cas9 to a specific genomic locus via complementary Watson-Crick base pairing adjacent to a Protospacer Adjacent Motif (PAM, specifically 5'-NGG-3' for SpCas9). Upon binding, Cas9 introduces a targeted double-strand break (DSB), triggering cellular repair mechanisms (NHEJ or HDR) to knock out or edit genes.

The Fundamental Engineering Dilemma: Designing an effective gRNA is exceptionally challenging because multiple competing biological objectives must be balanced simultaneously:
• **On-Target Efficiency (E):** The gRNA must efficiently bind the target sequence and stimulate cleavage.
• **Off-Target Specificity (S):** The gRNA must NOT bind or cleave unintentional sites across the 3.1 billion base pair human genome that possess 1, 2, or 3 mismatches.
• **Structural Suitability (R):** The single-stranded gRNA spacer must maintain an optimal melting temperature (T_m), avoid internal self-folding (low Minimum Free Energy / MFE), and maintain stable duplex thermodynamics (ΔG).

Central Problem Statement: Single-objective optimization fails because optimizing for efficiency alone often selects highly promiscuous guides that cause dangerous off-target mutations, while maximizing specificity alone can yield inactive guides. This project establishes a unified **End-to-End Multi-Objective Framework** that combines genome-wide alignment (Bowtie), thermodynamic feature modeling, weighted multi-objective scoring, and constrained Convex Linear Programming (CVXOPT) to select the mathematically optimal and biologically safe top gRNAs.

```
COMPLETE SYSTEM PIPELINE ARCHITECTURE:
Candidate gRNAs (2860) → Feature Extraction → GRCh38 Genome → Bowtie Alignment → Specificity Features → Weighted Multi-Objective Optimization → Feasibility Constraints Filtering → CVXOPT Convex LP Optimization → Selected gRNAs (5) → 2D RNA Structure Analysis → Interactive Dashboard
```

2. Dataset & Candidate gRNA Population

The project utilizes a candidate population of **2,860 synthetic 20-nt guide RNA sequences** stored in data/guide_candidates.csv (and detailed in data/gRNA_features_specificity.csv). These candidate sequences were extracted from a target human coding exon region adjacent to valid SpCas9 PAM motifs (5'-NGG-3'). Each record in the dataset captures the 20-bp spacer sequence, the 3-bp PAM motif, PAM genomic coordinate position, strand orientation (+/-), sequence length (20 nt), and nucleotide composition.

Distinction Between Project Stages:
1. *Candidate Generation:* Scanning the genomic target sequence for 5'-NGG-3' motifs to extract 20-nt spacer sequences.
2. *Feature Extraction:* Calculating biological, thermodynamic, structural, and genome-wide off-target metrics per sequence.
3. *Optimization:* Mathematical scoring (weighted sum) and constrained selection (CVXOPT Linear Program) across the multi-objective space.

3. Development Environment & WSL2 Integration

Bioinformatics alignment algorithms (such as Bowtie 1) are compiled natively for Linux POSIX environments. To execute Bowtie seamlessly on a Windows workstation without dual-booting or heavyweight virtual machines, **WSL2 (Windows Subsystem for Linux 2)** running **Ubuntu 22.04 LTS** was integrated into the Python execution pipeline.

Environment Interoperability & Integration:
• Windows host directory C:\Users\...\CRISPR-GuideRNA-Optimization is accessible inside Ubuntu via the DrvFs mount point /mnt/c/Users/....
• The core Python pipeline runs in a Windows Virtual Environment (venv), while feature_extraction.py dynamically translates Windows file paths to WSL POSIX paths using wslpath -u and invokes the Linux bowtie binary via subprocess.run(['wsl', 'bowtie', ...]).

4. Genome Alignment & Bowtie 1.3.1 Mechanics

Naive Python string matching (e.g. regex or Levenshtein distance) is computationally impossible for genome-wide off-target search, as it would require scanning 3.1 billion base pairs for 2,860 sequences across 4 mismatch levels (3.55×10^{13} operations). **Bowtie 1.3.1** was selected because it utilizes the *Burrows-Wheeler Transform (BWT)* and *Ferragina-Manzini (FM) index*, enabling ultrafast mismatch alignments ($O(m)$ time complexity for sequence length m) with minimal memory footprint (~2.2 GB RAM for human genome).

Installation & Version Verification Commands:

```
sudo apt update && sudo apt install bowtie -y
bowtie --version # Verified: Bowtie version 1.3.1 (64-bit)
```

5. Human Reference Genome (GRCh38.p14) & Indexing

To perform true genome-wide off-target search, the complete human reference genome **GRCh38.p14** (NCBI Assembly Accession: GCF_000001405.40, ~973 MB compressed, 3.1 GB uncompressed FASTA) was acquired via the NCBI Datasets CLI:

```
./datasets download genome accession GCF_000001405.40 --include genome
unzip ncbi_dataset.zip
```

Genome Indexing (bowtie-build): Bowtie cannot align reads directly against raw FASTA text files. The reference FASTA was transformed into a compressed 6-file Full-text index in Small Block (32-bit) format:

```
bowtie-build ncbi_dataset/data/GCF_000001405.40/GCF_000001405.40_GRCh38.p14_genomic.fna hg38_index
```

This generated six binary index files (hg38_index.1.ebwt through .4.ebwt, and reverse indexes .rev.1.ebwt, .rev.2.ebwt). The index prefix hg38_index is passed to Bowtie via the -x flag. Building the index is performed ONCE; subsequent alignment runs re-use the pre-built index prefix.

6. Feature Extraction Module (src/feature_extraction.py)

src/feature_extraction.py consumes the 2,860 candidate sequences, constructs a temporary query FASTA file, executes Bowtie inside WSL2, parses alignment results, calculates thermodynamic parameters, and exports data/gRNA_features_specificity.csv.

Feature Name	Biological Meaning	Pref.	Formula / Code Method
`gc_content`	Fraction of G and C nucleotides in guide	40-60%	(count('G')+count('C')) / length
`melting_temp_c`	Melting temperature (T_m) of DNA:RNA duplex	55-65°C	Nearest-Neighbor thermodynamics (Breslauer / SantaLucia)

doench2016_proxy_score	On-target cleavage efficiency proxy	Higher	Position-specific nucleotide preferences & homopolymer penalties
delta_g_kcal_mol	Duplex binding free energy (ΔG)	Lower	Nearest-Neighbor thermodynamic summation
folding_energy_kcal_mol	Secondary structure Minimum Free Energy (MFE)	Higher	RNAfold / dynamic programming loop free energy
off_target_count	Total off-target genomic alignments (0-3 mm)	Lower	Parsed from Bowtie default 8-column stdout
mismatch_score	Mean edit distance across off-target hits	Higher	Mean mismatches across off-targets (imputed to max if count=0)
off_target_capped	Boolean flag indicating if hits hit 1000 cap	False	off_target_count >= 999

Bowtie Alignment Subprocess Parameters & Output Parsing:

Bowtie is invoked with: bowtie -f -v 3 -k 1000 --best -p 4 -x hg38_index guides.fa
• -f: Input query is FASTA format.
• -v 3: Report alignments with up to 3 mismatches (no indels).
• -k 1000: Report up to 1000 valid alignments per guide (prevents memory explosion / returncode 9 on highly repetitive elements).
• --best: Report alignments ordered from best to worst edit distance.
• -x hg38_index: Path to the reference genome index.
Line-Ending Fix (WSL Pipe Artefact): Bowtie outputs 8 tab-separated fields. Field 7 (0-indexed) contains comma-separated mismatch descriptors (e.g. 4:T>A,18:A>C). For 0-mismatch alignments, Field 7 is empty. A stray \r injected by Windows subprocess pipe handling caused 0-mismatch lines to evaluate Field 7 as '\r', falsely splitting to 1 mismatch! Calling fields[7].strip() resolved this, restoring accurate 0-mismatch alignment counts.

7. Weighted Multi-Objective Optimization (src/optimization.py)

Single-objective ranking fails in guide design. src/optimization.py implements a multi-objective utility function combining Efficiency (E), Specificity (S), and Structural Suitability (R). Because raw features have incompatible physical units (e.g. T_m in °C, ΔG in kcal/mol, off-target counts in integers), all features are normalized to $[0, 1]$ before aggregation.

NORMALIZATION FORMULAS:
• Maximization Features (Doench Score, MFE, Mismatch Score): norm(x) = (x - min) / (max - min)
• Minimization Features (Off-Target Count, Duplex ΔG): norm(x) = (max - x) / (max - min)
• Plateau Target-Range Desirability (Melting Temp Tm): Derringer-Suich 1.0 inside [55, 65]°C, linear ramp to 0

SUB-OBJECTIVES & AGGREGATE UTILITY FUNCTION:
• Specificity Score: S_i = 0.70 · Burden_norm + 0.30 · Divergence_norm
• Structural Suitability: R_i = (Tm_norm + ΔG_norm + MFE_norm) / 3
• Overall Multi-Objective Score: F_i = 0.35 · E_i + 0.40 · S_i + 0.25 · R_i

8. Constrained Convex Optimization (src/convex_optimization.py)

Limitation of Unconstrained Weighted Ranking: Weighted sum ranking evaluates guides purely on relative score. A candidate can achieve a high weighted score (e.g. $F_i = 0.935$) yet violate vital biological boundaries (such as possessing 23 off-target sites or an extreme T_m). To guarantee safety, we formulate selection as a **Constrained Linear Program (LP)** — a fundamental class of Convex Optimization.

```
MATHEMATICAL FORMULATION OF CONSTRAINED LP:
Decision Variable:    x_i ∈ [0, 1]    (Selection allocation for guide i)
Objective Coefficient:    c_i = 0.35 E_i + 0.40 S_i + 0.25 R_i

Primal Maximization Problem:
     maximize    Σ_{i=1}^{n} c_i · x_i
     subject to    0 ≤ x_i ≤ u_i    ∀ i ∈ {1, ..., 2860}
                       Σ_{i=1}^{n} x_i ≤ K    (Budget limit K = 10)

Biological Feasibility Indicator (u_i):
     u_i = 1    iff    [0.40 ≤ GC_i ≤ 0.60]  ∧  [55.0 ≤ Tm_i ≤ 65.0 °C]  ∧  [off_target_count_i ≤
20]  ∧  [off_target_capped_i = False]
     u_i = 0    otherwise (hard exclusion from feasible region)
```

Mathematical Proof of Convexity for Viva Defense:

1. *Linearity of Objective:* The objective function $f(x) = c^T x = \sum c_i x_i$ is linear. Linear functions satisfy $f(\lambda x + (1-\lambda)y) = \lambda f(x) + (1-\lambda)f(y)$, meaning they are simultaneously convex and concave.
2. *Convexity of Feasible Region:* Every inequality constraint $-x_i \le 0$, $x_i \le u_i$, and $\sum x_i \le K$ defines a closed half-space $H_k = \{x \in \mathbb{R}^n : g_k(x) \le 0\}$. Half-spaces are convex sets. The feasible region $\mathcal{S} = ■igcap_{k} H_k$ is an intersection of convex sets, which is mathematically guaranteed to be a convex polytope.
3. *Convex Optimization Definition:* A problem is a Convex Optimization Problem if the objective function to be minimized is convex and the feasible set is convex. Since Linear Programming minimizes $-c^T x$ over a convex polytope $\mathcal{S}$, it is a rigorously defined convex optimization problem.
4. *Polytope Vertex Optimal Solution:* Fundamental Theorem of LP guarantees that the optimal solution of an LP over a bounded convex polytope occurs at a extreme point (vertex). Because box constraints are $x_i \in [0, 1]$ and $u_i \in \{0, 1\}$, the vertices are integer binary vectors $x^* \in \{0, 1\}^n$, ensuring exact 0/1 guide selection without requiring NP-hard integer programming.

9. CVXOPT Solver Implementation & Empirical Results

The Python package CVXOPT solves LPs in standard canonical form:

minimize $q^T x$ **subject to** $G x \le h$

Maximizing $c^T x$ is converted by setting $q = -c$. Matrix G of shape $(2n+1) imes n$ and vector h of length $2n+1$ encode lower bounds ($-x_i \le 0$), upper bounds ($x_i \le u_i$), and total budget ($\sum x_i \le K=10$).

Rank	Guide Sequence (5' → 3')	Score (c_i)	E_i	S_i	R_i	GC%	T_m (°C)	Off-Targets
#1	GTACGCTTTAGGCCGGCTCA	**0.8681**	0.732	1.000	0.848	60.0%	56.72	9
#2	GCTGAACGTCGTGAAGCGGA	**0.8554**	0.732	0.998	0.800	60.0%	57.61	12
#3	ACGCTACCATGGCGTGAGAC	**0.8385**	0.732	0.996	0.736	60.0%	56.74	15
#4	CTGAACGTCGTGAAGCGGAA	**0.7979**	0.557	0.989	0.830	55.0%	55.21	15
#5	TAGGCATCACGGAGCGGTTA	**0.7934**	0.557	0.977	0.831	55.0%	55.10	17

Empirical Filtering Breakdown (2,860 Candidates):
• Total candidates: 2,860 | GC Pass [0.40-0.60]: 1,708 | T_m Pass [55-65°C]: 991 | Off-Target ≤ 20: 15 | Uncapped: 1,378
• **ALL Constraints Feasible: Exactly 5 gRNAs**. Since $5 < K=10$, CVXOPT selected all 5 feasible guides ($x_i^* = 1.0$, total objective = 4.1532).

10. Research Dashboard & RNA Structure Visualization

An interactive research-grade dashboard was created at dashboard/index.html (generated by src/generate_dashboard.py and served via src/run_dashboard.py). The dashboard features 4 dedicated analytical tabs:
1. **Overview Tab:** Pipeline architecture Sankey flow, KPI metrics, and summary table of selected guides.
2. **Optimization Tab:** Constraint filtering funnel, efficiency vs. specificity scatter plot with feasibility region, weighted vs. CVXOPT comparison, and sub-objective bar chart.
3. **gRNA Explorer Tab:** Deep-dive candidate viewer with sequence details,PAM, score breakdown, constraint verification matrix, and explicit 'Why Selected?' analytical rationale.
4. **RNA Structure Tab:** 2D secondary structure visualizer rendering nucleotide base badges (A, C, G, U/T), dot-bracket notation strings (e.g. ....(((.....)))), Minimum Free Energy (MFE), duplex ΔG, paired base fraction, and a dedicated 3D structure prediction extension placeholder (explaining future RNAComposer/AlphaFold 3 integration).

11. Biology vs. Mathematics vs. Computing Cross-Mapping

Pipeline Stage	Biological Concept	Mathematical Formulation	Computing / Implementation
Candidate Gen.	CRISPR target exon & SpCas9 PAM (5'-NGG-3')	Substring pattern matching over sequence space	Biopython sequence parsing
Off-Target Mapping	Genome-wide off-target cleavage risk	Hamming distance / Edit distance $d(g_i, s_j) \le 3$	Bowtie 1.3.1 FM-Index alignment in WSL2
Feature Extraction	Duplex thermodynamics, MFE, cleavage proxy	Nearest-Neighbor ΔG, MFE loop energy, T_m	feature_extraction.py & NumPy
Multi-Obj. Scoring	Trade-off between efficiency, safety, stability	Min-max scaling & weighted sum $c_i = \sum w_k f_k$	optimization.py & pandas DataFrames
Convex Optimization	Strict biological safety & activity bounds	Constrained Linear Program: $\max c^T x$ s.t. $G x \le h$	convex_optimization.py & CVXOPT solver
Visualization	Structural 2D base-pairing & safety verification	Graph arc diagrams, scatter plots, funnel waterfall	Matplotlib, Plotly.js, interactive HTML dashboard

12. Viva Defense Guide & Frequently Asked Questions

Q1: Why is this project classified as Convex Optimization?

Answer: The objective function f(x) = c^T x is linear, which is both convex and concave. The feasible region S is formed by the intersection of linear inequality constraints (-x_i <= 0, x_i <= u_i, sum x_i <= K), forming a convex polytope. Minimizing -c^T x over a convex polytope satisfies the mathematical definition of a Convex Optimization problem.

Q2: Why did you use Linear Programming (LP) instead of Integer Linear Programming (ILP)?

Answer: By the Fundamental Theorem of Linear Programming, the optimal solution of an LP over a bounded convex polytope is attained at one of its extreme points (vertices). Since the box constraints are x_i in [0, 1] and u_i in {0, 1}, the vertices of the feasible polytope are binary vectors x* in {0, 1}^n. LP relaxation guarantees 0/1 binary selection while maintaining polynomial-time solvability via interior-point methods.

Q3: Why did unconstrained weighted optimization select guides that failed biological constraints?

Answer: Weighted sum optimization ranks guides purely on aggregate relative score without hard cutoffs. A candidate with a high efficiency score can mask an unacceptably high off-target count or bad Tm. Constrained optimization introduces hard indicator bounds (u_i), restricting the search strictly to biologically viable candidates before maximizing the objective.

Q4: How did you handle the Bowtie reporting cap (-k 1000) in the off-target score?

Answer: An off-target count of 999 indicates the guide reached Bowtie's reporting limit, meaning the true count is >= 999. In optimization.py, capped guides (off_target_capped == True) have their raw count set to the dataset maximum before normalization and hard-floored to 0.0 burden score post-normalization, ensuring truncated counts are not mistaken for exact values.

Q5: What is the significance of the 2,860 to 5 candidate filtering?

Answer: Out of 2,860 raw candidates, 1,708 pass GC content (40-60%), 991 pass Tm (55-65°C), 15 pass off-target count (<= 20), and 1,378 are uncapped. Only 5 guides satisfy ALL four biological constraints simultaneously, demonstrating the extreme stringency of multi-objective genome editing criteria.

13. Reproduction Sequence & Conclusion

Complete End-to-End Command Sequence:

```
# 1. Feature Extraction & Bowtie Alignment inside WSL2
python src/feature_extraction.py --input data/guide_candidates.csv --output
data/gRNA_features_specificity.csv --bowtie-index data/hg38_index/hg38_index --max-mismatches 3
--threads 4

# 2. Weighted Multi-Objective Optimization
python src/optimisation.py --input data/gRNA_features_specificity.csv --output
data/optimized_guides.csv

# 3. Constrained Convex LP Optimization
python src/convex_optimization.py --input data/gRNA_features_specificity.csv --output
data/convex_optimized_guides.csv

# 4. Visualization & Comparative Figures
python src/convex_visualization.py
python src/create_weighted_vs_convex_selection.py

# 5. Generate & Launch Interactive Research Dashboard
python src/generate_dashboard.py
python src/run_dashboard.py
```

Summary Conclusion: This framework successfully bridges molecular biology, computational alignment, and mathematical convex optimization. By replacing naive single-objective or unconstrained ranking with a mathematically rigorous Linear Program (CVXOPT), the pipeline guarantees biological feasibility (GC, T_m, off-target safety) while maximizing overall guide RNA utility.